

Breakout Game — Deep Dive

Explainer document (generated for the project owner)

Contents

1	What this project is	1
1.1	What you can do when you run it	2
2	Module architecture	2
3	The screen and the state machine	3
3.1	State machine	3
3.2	The main loop, frame by frame	4
4	paddle.py: the paddle	4
5	ball.py: the ball and the difficulty ramp	4
5.1	Bouncing	5
5.2	Difficulty ramp	5
5.3	Resets	5
6	blocks.py: the wall, its layout, and the break effects	6
6.1	Procedural layout	6
6.2	Collision detection by distance	6
6.3	Break effects: flash and particle burst	6
6.4	Reset	7
7	scoreboard.py: score, lives, and persistence	7
8	Persistence: highestScore.txt	8
9	Design decisions and known limitations	8
10	Glossary	8

1 What this project is

This is a playable clone of **Breakout**, the classic arcade game where a paddle bounces a ball into a wall of blocks, built entirely with Python’s built-in `turtle` module. `turtle` is a standard-library graphics package that draws by moving a virtual “turtle” cursor around a window; it is normally used to teach programming, but here it is repurposed as a minimal 2D game-rendering surface.

There is no external game engine (no Pygame, no Unity, nothing installed beyond Python itself) — every piece of motion, collision, and drawing is written by hand.

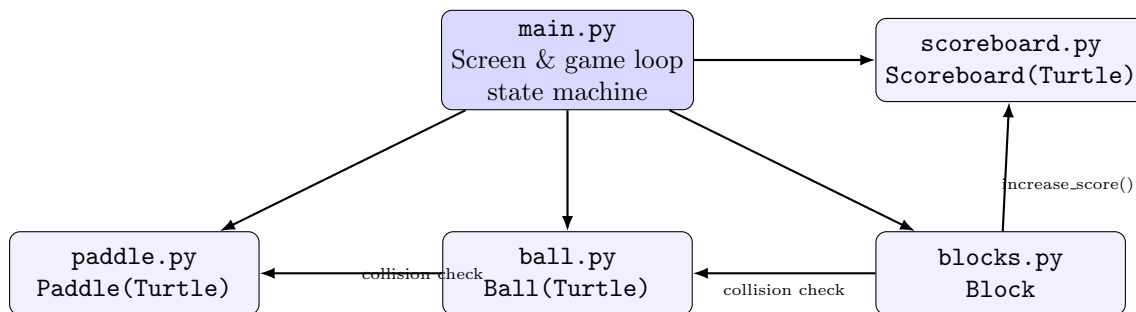
A **game loop** is the core pattern behind nearly all real-time games: a piece of code that runs over and over, as fast as the screen can be redrawn, each pass moving objects a small step, checking for collisions, and redrawing the screen. This project’s game loop lives in `main.py`.

1.1 What you can do when you run it

- Move a paddle left/right with the arrow keys.
- Watch a ball bounce off the four walls, the paddle, and the blocks.
- Break blocks, each one making a short white flash and firing a handful of small particles outward before disappearing.
- Lose one of three lives every time the ball drops past the paddle; the ball re-centers and keeps going until lives run out.
- Feel the ball speed up gradually the longer a rally lasts.
- See a start screen, and a game-over or you-win screen with the score and a restart prompt.
- Keep a high score that survives between runs, stored in a plain text file, `highestScore.txt`.

2 Module architecture

The project is split into five files, each owning one concern. This is a common design pattern called **separation of concerns**: instead of one large file mixing paddle movement, ball physics, block layout, and score tracking together, each responsibility gets its own file and, where it makes sense, its own class.



`main.py` is the only file that imports all the others; it creates one instance of each class and calls their methods every frame. None of the other four modules import each other directly — `ball.py` does not know `blocks.py` exists, for example. `main.py` passes objects as arguments instead (e.g. `block.check_collision.ball(ball, scoreboard)` hands the ball and scoreboard objects into the blocks module), which is a form of **dependency injection**: a module receives the objects it needs to act on as parameters, rather than reaching out and importing them itself.

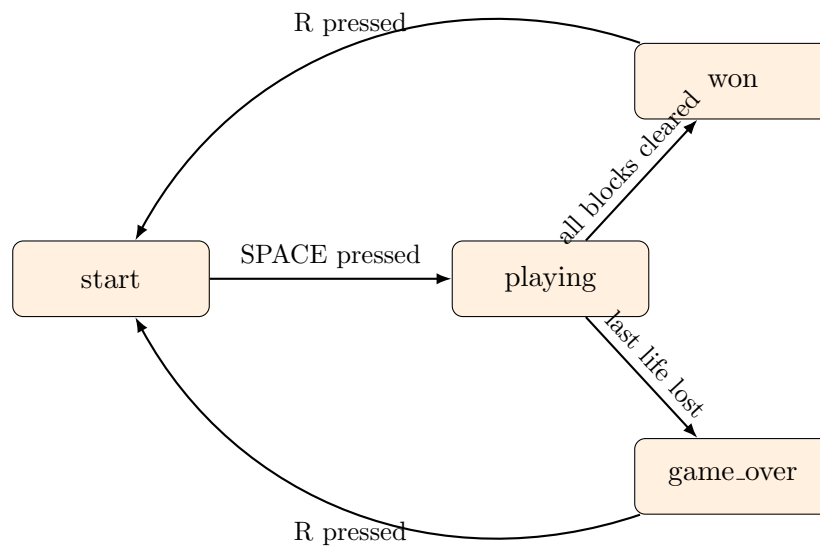
Four of the five classes (Paddle, Ball, Scoreboard, and each individual block) **inherit from Turtle** — inheritance means the class is built on top of `turtle.Turtle` and automatically gets its drawing, movement, and color methods for free, instead of reimplementing them.

3 The screen and the state machine

`main.py` sets up an 800×600 black `Screen` and turns on manual updates: `wn.tracer(0)` tells turtle *not* to redraw the window after every single drawing command (its default, which is slow and flickery), and `wn.update()` is called once per loop iteration instead, redrawing everything in one batch. This is exactly the change needed to turn `turtle` from a teaching toy into something that behaves like a real animation loop.

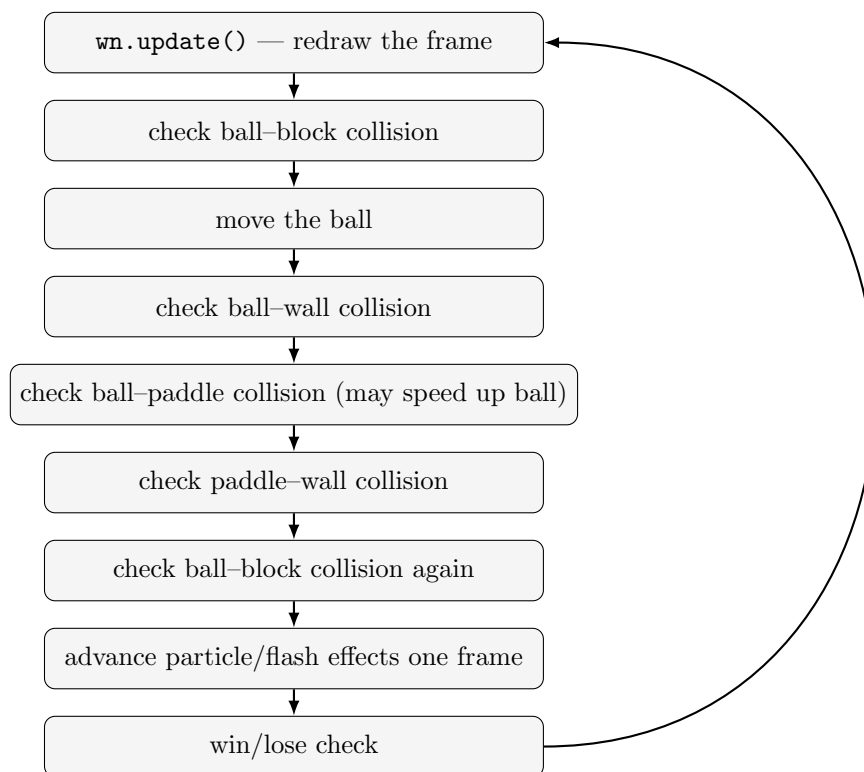
3.1 State machine

A **state machine** is a model where a system is always in exactly one named “state” at a time, and specific events move it to another state. This game has four states, held in the plain string variable `game_state`:



Physics and collision checks in the main loop only run while `game_state == "playing"`; in every other state the loop just waits for a keypress (`space` to start, `r` to restart), except that it still calls `block.update_effects()` so an in-flight particle burst finishes animating instead of freezing mid-air the instant the round ends.

3.2 The main loop, frame by frame



The window close event raises a `Terminator` exception inside `turtle`; `main.py` wraps the whole loop in `try/except Terminator: pass` so closing the window ends the process quietly instead of printing a traceback.

4 paddle.py: the paddle

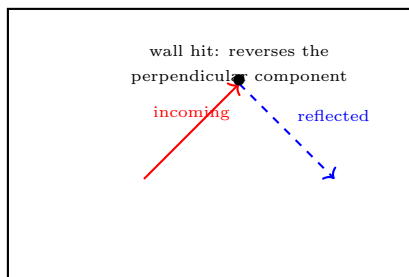
The simplest module. `Paddle` inherits from `Turtle`, is drawn as a stretched white square (`shapesize(stretch_wid=stretch_len=8)`), and starts at $(0, -250)$.

- `paddle_right()` / `paddle_left()` move it ± 20 pixels on the x-axis; bound to the Right/Left arrow keys in `key_bindings()`.
- `check_collision.wall()` clamps the paddle's x position to $[-320, 320]$ so it cannot be pushed off-screen.

5 ball.py: the ball and the difficulty ramp

`Ball` inherits from `Turtle`, drawn as a white circle, and tracks its own velocity as two numbers, `dx` and `dy` (change in x and y per frame). Moving the ball is just `goto(xcor()+dx, ycor()+dy)` every frame — there is no physics engine, gravity, or acceleration beyond what the code adds explicitly.

5.1 Bouncing

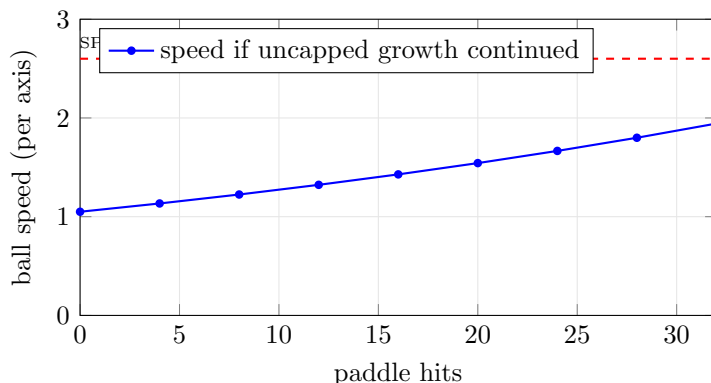


`check_collision_wall()` checks the ball’s position against all four screen edges (± 390 x, ± 290 y). Hitting a vertical edge snaps the ball back inside and flips `dx` (`bounce_x()`); a horizontal edge flips `dy` (`bounce_y()`). This is a simplified bounce: real physics would reflect the velocity vector around the surface normal, but since the walls here are always axis-aligned, flipping one component is equivalent and far simpler.

`check_collision_paddle()` only registers a bounce when the ball is below $y = -240$ and within ± 80 pixels of the paddle’s x position — this direction sensitivity matters because without it, the ball could “bounce” off the paddle’s invisible vertical extent even while approaching from the side or from above, which would look wrong.

5.2 Difficulty ramp

Every time the ball is returned by the paddle, an internal counter `paddle_hits` increments. Every `HITS_PER_STEP` (4) hits, `speed_up()` multiplies both `dx` and `dy` by `SPEED_STEP` (1.08), preserving direction (sign) but increasing magnitude, capped at `MAX_SPEED` (2.6) so the game never becomes literally unplayable.



BASE_SPEED is 1.05 per axis per frame; the initial and immediately subsequent hit counts (0, 4, 8, 12...) are exact (1.05×1.08^n) until the cap intervenes at higher hit counts.

5.3 Resets

Two different resets exist because two different situations need different behavior:

- `reset_after_miss()`: called when a life is lost but the player still has lives left. Re-centers the ball and picks a new random horizontal direction, but *keeps* the current speed (the difficulty ramp is not undone by a single miss).

- `full_reset()`: called only on a full restart after a finished game. Resets position *and* speed back to `BASE.SPEED` *and* zeroes `paddle_hits`, so a new game starts at the original difficulty.

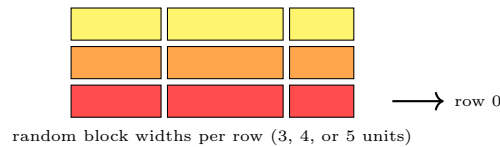
`check_game_over()` returns true once the ball drops below $y = -280$, i.e. past the paddle's row entirely.

6 blocks.py: the wall, its layout, and the break effects

`Block` is not itself a `Turtle` subclass; it is a manager object that owns a list of individual block `Turtle` instances plus bookkeeping lists for in-flight visual effects.

6.1 Procedural layout

The wall is built one block at a time by `make_block()` / `place_block()`, packing blocks left to right in a row and wrapping to a new row when the next block would run past the left edge, stopping entirely once there is no more vertical room (`can_make_more` flips to `False`).

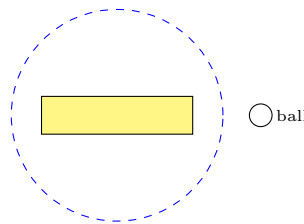


Each block's width is randomly chosen (`random.choice([3,4,5])`) and scaled to pixels via `shapeseize`. The row a block belongs to is computed from its final y-position, and `ROW_COLORS` assigns one fixed color per row (cycling if there were ever more rows than colors), so the finished wall reads as horizontal color bands instead of a random speckle of colors.

6.2 Collision detection by distance

`check_collision_ball()` does not use rectangle-vs-circle collision math; instead it checks `block.distance(ball) < 35`, i.e. a fixed-radius circle around the block's center point.

collision radius (35px), same for every block

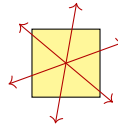


This is an approximation: a block that is 5 units wide and one that is 3 units wide are both treated as the same size for collision purposes, so wide blocks are effectively slightly harder to hit at their far edges than a true bounding-box check would produce. It is cheap to compute and visually close enough at this block size, which is why the project keeps it (see [Section 9](#)).

6.3 Break effects: flash and particle burst

When a block is hit, three things happen at once:

1. The block is removed from `all_blocks` (so it can no longer be collided with or counted toward the win condition) and recolored white, then added to a `fading` list with a frame countdown (`FADE_FRAMES = 3`).
2. `spawn_burst()` creates `BURST_PARTICLES` (5) tiny circular turtles at the block's former position, each given a random outward direction (angle picked uniformly 0–360°) and a fixed speed, tracked with a countdown (`BURST_LIFETIME = 8` frames).
3. The ball's `dy` is flipped, bouncing it away from the block.



particles fly outward from the broken block's position

`update_effects()` must be called once per frame (from `main.py`, in both the `playing` state and every other state) to advance each fading block and particle by one frame, hiding them once their countdown reaches zero. This is why particle bursts keep animating briefly even after the round ends instead of freezing mid-burst.

6.4 Reset

`reset()` hides every current block and any in-flight effect, clears all internal lists, and calls `make_all_blocks()` again to build a fresh wall — used when the player restarts after a finished game.

7 scoreboard.py: score, lives, and persistence

`Scoreboard` inherits from `Turtle` but is only ever used to render text (`hideturtle()` is called immediately so the turtle cursor itself is invisible).

- At *module import time* (before any class is even instantiated), the module tries to read `highestScore.txt`. If the file is missing, it creates it with 0; if the contents aren't a valid number, it falls back to 0 in memory. This means importing `scoreboard.py` has a side effect (it may read and write a file) purely as a consequence of when this code runs, not inside any function.
- `increase_score()`: +1 to score each block broken; bumps the in-memory high score too if the new score exceeds it.
- `lose_life()`: -1 life on a miss; returns `True` once lives reach zero, which `main.py` uses to decide whether to end the game or just respawn the ball.
- `save_high_score()`: writes the current high score to disk. Called whenever a game ends (win or loss), not on every point scored.
- `new_round()`: resets score and lives for a fresh game while leaving the persisted high score untouched.

`STARTING_LIVES = 3` is the only place the starting life count is defined.

8 Persistence: `highestScore.txt`

The high score lives in a plain text file in the project directory containing a single integer, written with Python's built-in `open(...).write(...)` — no database, no serialization format, just a number as text. It is read once at import time and rewritten whenever a round ends.

9 Design decisions and known limitations

These are documented directly in the project's own README as deliberate, acknowledged trade-offs, not oversights:

- **Distance-based collision** is a reasonable stand-in for real hitbox math only while every object is roughly circular or small relative to the play field; it stops being accurate once object sizes vary a lot, which is visible in the block collision radius being a single constant (35px) regardless of a block being 3, 4, or 5 units wide.
- **The main loop calls `block.check_collision_ball()` twice per frame** (once before moving the ball, once after), apparently to reduce the chance of the ball tunneling through a block between checks at higher speeds. It works, but runs the check twice as often as strictly necessary most of the time.
- **Reading the high-score file at module-import time** rather than inside `__init__` or a dedicated loader function works because the module is only ever imported once in this program, but it would not survive being imported twice, or tested in isolation, without surprising file I/O.

10 Glossary

turtle

Python's standard-library graphics module; draws by moving a virtual cursor around a window.

game loop

Code that repeatedly moves objects, checks collisions, and redraws the screen, many times per second.

state machine

A model where a system is always in exactly one named state, moving between states on specific events.

inheritance

A class built on top of another class, automatically gaining its behavior (here, every game object inherits turtle's drawing and movement methods).

dependency injection

Passing an object as a parameter to code that needs it, instead of that code importing/creating the object itself.

separation of concerns

Splitting a program so each file/module owns one responsibility.